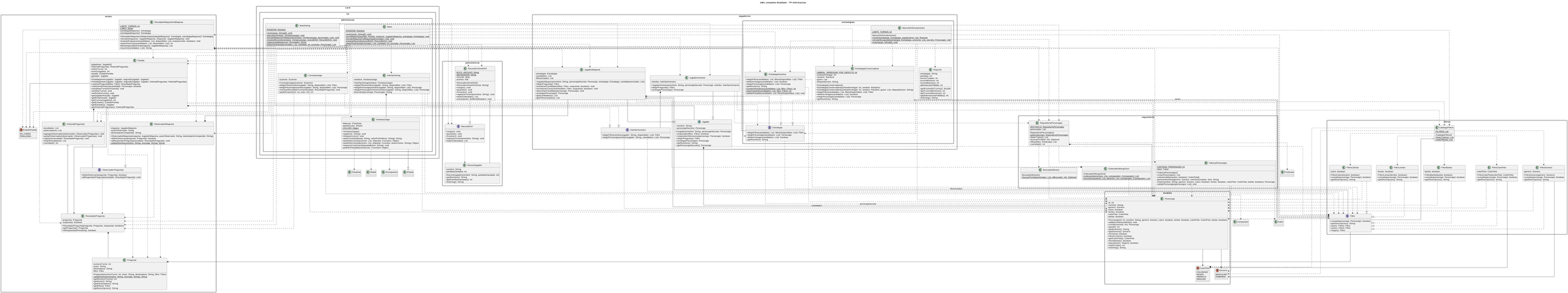

UML completo detallado - TP Adivinanzas

com
tp
adivinanzas
MainSwing
-RANDOM: Random
+main(args: String[]): void
-ejecutar(ventana: VentanaJuego): void
-simularMaquinaVsMaquina(ventana: VentanaJuego, personajes: List): void
-mostrarRecords(ventana: VentanaJuego, recordDAO: RecordDAO): void
-capturarSalida(accion: Runnable): String
-elegirDistintos(personajes: List, cantidad: int, excluido: Personaje): List
Main
-RANDOM: Random
+main(args: String[]): void
-turnoMaquina(partida: Partida, maquina: JugadorMaquina, estrategia: Estrategia): void
-simularMaquinaVsMaquina(personajes: List): void
-mostrarRecords(recordDAO: RecordDAO): void
-elegirDistintos(personajes: List, cantidad: int, excluido: Personaje): List
ConsolaJuego
-scanner: Scanner
+ConsolaJuego(scanner: Scanner)
+elegirFiltro(nombreJugador: String, disponibles: List): Filtro
+elegirPersonaje(nombreJugador: String, disponibles: List): Personaje
+mostrarResultadoTurno(resultado: ResultadoPregunta): void
+pedirEntero(min: int, max: int): int
InterfazSwing
-ventana: VentanaJuego
+InterfazSwing(ventana: VentanaJuego)
+elegirFiltro(nombreJugador: String, disponibles: List): Filtro
+elegirPersonaje(nombreJugador: String, disponibles: List): Personaje
+elegirPersonajeOVolver(nombreJugador: String, disponibles: List): Personaje
-describir(personaje: Personaje): String
VentanaJuego
-bitacora: JTextArea
-panelAccion: JPanel
-VOLVER: Object
+VentanaJuego()
+log(texto: String): void
+limpiarAccion(): void
+pedirTexto(etiqueta: String, valorPorDefecto: String): String
+pedirEleccion(opciones: List, etiqueta: Function): Object
+pedirEleccion(opciones: List, etiqueta: Function, botonVolver: String): Object
+esperarContinuar(etiquetaBoton: String): void
-pedirEnPantalla(constructor: Function): Object
JTextArea
JPanel
JComponent
JFrame
motor
SimuladorMaquinaVsMaquina
-LIMITE_TURNOS: int
-LINEA: String
-estrategiaMaquina1: Estrategia
-estrategiaMaquina2: Estrategia
+SimuladorMaquinaVsMaquina(estrategiaMaquina1: Estrategia, estrategiaMaquina2: Estrategia)
+simular(maquina1: JugadorMaquina, maquina2: JugadorMaquina): void
-mostrarEvaluacion(candidatos: List, disponibles: List, evaluaCortes: boolean): void
-mejorPeorCaso(candidatos: List, disponibles: List): int
-filtrosDisponiblesPara(maquina: JugadorMaquina): List
-resumir(candidatos: List): String
Partida
-jugadores: Jugador[]
-historialPreguntas: HistorialPreguntas
-indiceTurno: int
-turnosJugados: int
-estado: EstadoPartida
-ganador: Jugador
+Partida(primerJugador: Jugador, segundoJugador: Jugador)
+Partida(primerJugador: Jugador, segundoJugador: Jugador, historialPreguntas: HistorialPreguntas)
+realizarPregunta(filtro: Filtro): ResultadoPregunta
+realizarAdivinanza(candidato: Personaje): boolean
-completarTurnoSinVictoria(): void
-cambiarTurno(): void
-verificarEnCurso(): void
+getJugadorActual(): Jugador
+getRivalActual(): Jugador
+getTurnosJugados(): int
+getEstado(): EstadoPartida
+getGanador(): Jugador
+getHistorialPreguntas(): HistorialPreguntas
EstadoPartida
EN_CURSO
FINALIZADA
HistorialPreguntas
-resultados: List
-observadores: List
+agregarObservador(observador: ObservadorPreguntas): void
+quitarObservador(observador: ObservadorPreguntas): void
+registrar(resultado: ResultadoPregunta): void
+listarResultados(): List
+cantidad(): int
ObservadorMaquina
-maquina: JugadorMaquina
-autorObservado: String
-destinatarioCompartido: String
+ObservadorMaquina(maquina: JugadorMaquina, autorObservado: String, destinatarioCompartido: String)
+debeObservar(pregunta: Pregunta): boolean
+alResponderPregunta(resultado: ResultadoPregunta): void
-validarNombre(nombre: String, mensaje: String): String
ObservadorPreguntas
+debeObservar(pregunta: Pregunta): boolean
+alResponderPregunta(resultado: ResultadoPregunta): void
ResultadoPregunta
-pregunta: Pregunta
-respuesta: boolean
+ResultadoPregunta(pregunta: Pregunta, respuesta: boolean)
+getPregunta(): Pregunta
+isRespuestaAfirmativa(): boolean
Pregunta
-numeroTurno: int
-autor: String
-destinatario: String
-filtro: Filtro
+Pregunta(numeroTurno: int, autor: String, destinatario: String, filtro: Filtro)
-validarNombre(nombre: String, mensaje: String): String
+getNumeroTurno(): int
+getAutor(): String
+getDestinatario(): String
+getFiltro(): Filtro
+getDescripcion(): String
persistencia
RecordArchivoDAO
-RUTA_ARCHIVO: String
-SEPARADOR: String
-records: Map
-archivo: File
+RecordArchivoDAO()
+RecordArchivoDAO(ruta: String)
+cargar(): void
+guardar(): void
+mostrar(): void
+registrarVictoria(nombre: String): void
+listarOrdenado(): List
-cerrar(lector: BufferedReader): void
RecordDAO
+cargar(): void
+guardar(): void
+mostrar(): void
+registrarVictoria(nombre: String): void
+listarOrdenado(): List
RecordJugador
-nombre: String
-partidasGanadas: int
+RecordJugador(nombre: String, partidasGanadas: int)
+getNombre(): String
+getPartidasGanadas(): int
+toString(): String
jugadores
JugadorMaquina
-estrategia: Estrategia
-candidatos: List
-filtrosUsados: List
+JugadorMaquina(nombre: String, personajeSecreto: Personaje, estrategia: Estrategia, candidatosIniciales: List)
+elegirPregunta(): Filtro
+actualizarCandidatos(filtro: Filtro, respuesta: boolean): void
+incorporarConocimiento(filtro: Filtro, respuesta: boolean): void
+descartarCandidato(personaje: Personaje): void
+arriesgarPersonaje(): Personaje
+getCandidatos(): List
+getFiltrosUsados(): List
JugadorHumano
-interfaz: InterfazHumano
+JugadorHumano(nombre: String, personajeSecreto: Personaje, interfaz: InterfazHumano)
+elegirPregunta(): Filtro
+arriesgarPersonaje(): Personaje
InterfazHumano
+elegirFiltro(nombreJugador: String, disponibles: List): Filtro
+elegirPersonaje(nombreJugador: String, candidatos: List): Personaje
Jugador
-nombre: String
-personajeSecreto: Personaje
#Jugador(nombre: String, personajeSecreto: Personaje)
+responder(filtro: Filtro): boolean
+responderAdivinanza(personaje: Personaje): boolean
+elegirPregunta(): Filtro
+arriesgarPersonaje(): Personaje
+getNombre(): String
+getPersonajeSecreto(): Personaje
estrategias
BancoDeSimulaciones
-LIMITE_TURNOS: int
-BancoDeSimulaciones()
+medir(estrategia: Estrategia, repeticiones: int): Reporte
-simularBusqueda(estrategia: Estrategia, universo: List, secreto: Personaje): int[]
+main(args: String[]): void
EstrategiaAsertiva
+elegirFiltro(candidatos: List, filtrosDisponibles: List): Filtro
+debeArriesgar(candidatos: List): boolean
+elegirPersonaje(candidatos: List): Personaje
+getNombre(): String
+contarAfirmativos(candidatos: List, filtro: Filtro): int
+peorCasoDe(candidatos: List, filtro: Filtro): int
-validarEntrada(candidatos: List, filtrosDisponibles: List): void
EstrategiaConservadora
-UMBRAL_ARRIESGAR_POR_DEFECTO: int
-umbralArriesgar: int
-random: Random
-guion: List
-etiquetaGuion: String
+EstrategiaConservadora()
+EstrategiaConservadora(umbralArriesgar: int, random: Random)
+EstrategiaConservadora(umbralArriesgar: int, random: Random, guion: List, etiquetaGuion: String)
+elegirFiltro(candidatos: List, filtrosDisponibles: List): Filtro
+debeArriesgar(candidatos: List): boolean
+elegirPersonaje(candidatos: List): Personaje
+getNombre(): String
Reporte
-estrategia: String
-partidas: int
-turnosTotales: int
-turnosMinimos: int
-turnosMaximos: int
-adivinanzasFallidas: int
+getPromedioTurnos(): double
+getTurnosMinimos(): int
+getTurnosMaximos(): int
+getAdivinanzasFallidas(): int
+toString(): String
Estrategia
+elegirFiltro(candidatos: List, filtrosDisponibles: List): Filtro
+elegirPersonaje(candidatos: List): Personaje
+debeArriesgar(candidatos: List): boolean
+getNombre(): String
guion
repositorio
RepositorioPersonajes
-INSTANCIA: RepositorioPersonajes
-personajes: List
-RepositorioPersonajes()
+getInstancia(): RepositorioPersonajes
+listarTodos(): List
+buscarPorId(id: int): Optional
+filtrar(filtro: Predicate): List
+cantidad(): int
BuscadorBinario
-BuscadorBinario()
+buscarPorId(personajes: List, idBuscado: int): Optional
OrdenadorMergeSort
-OrdenadorMergeSort()
+ordenar(elementos: List, comparador: Comparator): List
-mezclar(izquierda: List, derecha: List, comparador: Comparator): List
FabricaPersonajes
-CANTIDAD_PERSONAJES: int
-faker: Faker
+FabricaPersonajes()
+crearPersonajes(): List
-coloresValidos(calvo: boolean): ColorPelo[]
-generarNombre(genero: Genero, nombresUsados: Set): String
-crear(nombre: String, genero: Genero, calvo: boolean, lentes: boolean, colorPelo: ColorPelo, barba: boolean): Personaje
-validarPersonajes(personajes: List): void
Predicate
filtrosUsados
personajeSecreto
candidatos
modelo
Personaje
-id: int
-nombre: String
-genero: Genero
-calvo: boolean
-lentes: boolean
-colorPelo: ColorPelo
-barba: boolean
+Personaje(id: int, nombre: String, genero: Genero, calvo: boolean, lentes: boolean, colorPelo: ColorPelo, barba: boolean)
-validarCoherenciaPelo(): void
+conId(nuevoId: int): Personaje
+getId(): int
+getNombre(): String
+getGenero(): Genero
+isCalvo(): boolean
+tieneLentes(): boolean
+getColorPelo(): ColorPelo
+tieneBarba(): boolean
+equals(otro: Object): boolean
+hashCode(): int
+toString(): String
ColorPelo
COLORADO
NEGRO
AMARILLO
NINGUNO
Genero
MASCULINO
FEMENINO
Comparator
Faker
filtros
CatalogoFiltros
-FILTROS: List
-CatalogoFiltros()
+listarTodos(): List
-crearFiltros(): List
FiltroCalvicie
-calvo: boolean
+FiltroCalvicie(calvo: boolean)
+cumple(personaje: Personaje): boolean
+getDescripcion(): String
FiltroLentes
-lentes: boolean
+FiltroLentes(lentes: boolean)
+cumple(personaje: Personaje): boolean
+getDescripcion(): String
FiltroBarba
-barba: boolean
+FiltroBarba(barba: boolean)
+cumple(personaje: Personaje): boolean
+getDescripcion(): String
FiltroColorPelo
-colorPelo: ColorPelo
+FiltroColorPelo(colorPelo: ColorPelo)
+cumple(personaje: Personaje): boolean
+getDescripcion(): String
FiltroGenero
-genero: Genero
+FiltroGenero(genero: Genero)
+cumple(personaje: Personaje): boolean
+getDescripcion(): String
Filtro
+cumple(personaje: Personaje): boolean
+getDescripcion(): String
+y(otro: Filtro): Filtro
+o(otro: Filtro): Filtro
+negar(): Filtro